



UNIVERSITÀ DEGLI STUDI
DI SALERNO

ReFAIR

Corso di Laurea Magistrale in Informatica

Corso di Ingegneria, gestione ed evoluzione del software
aa 2024/2025

Docente

Prof. Andrea De Lucia

Tutor progetto

Dott. Gianmario Voria

Studenti

Carmine Pastore - 0522501437

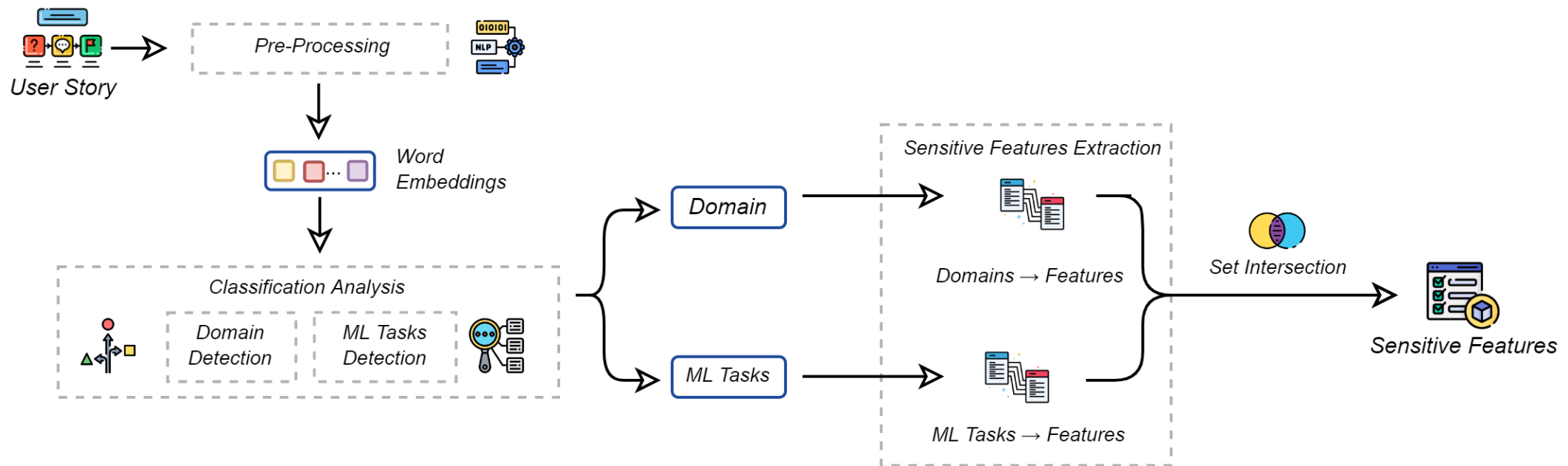
Nicola Laurino - 0522501437

Indice

- Introduzione
- Sistema attuale
- Reverse engineering
- CR1
- CR2
- CR3
- TESTING
- Sistema post modifiche

Introduzione

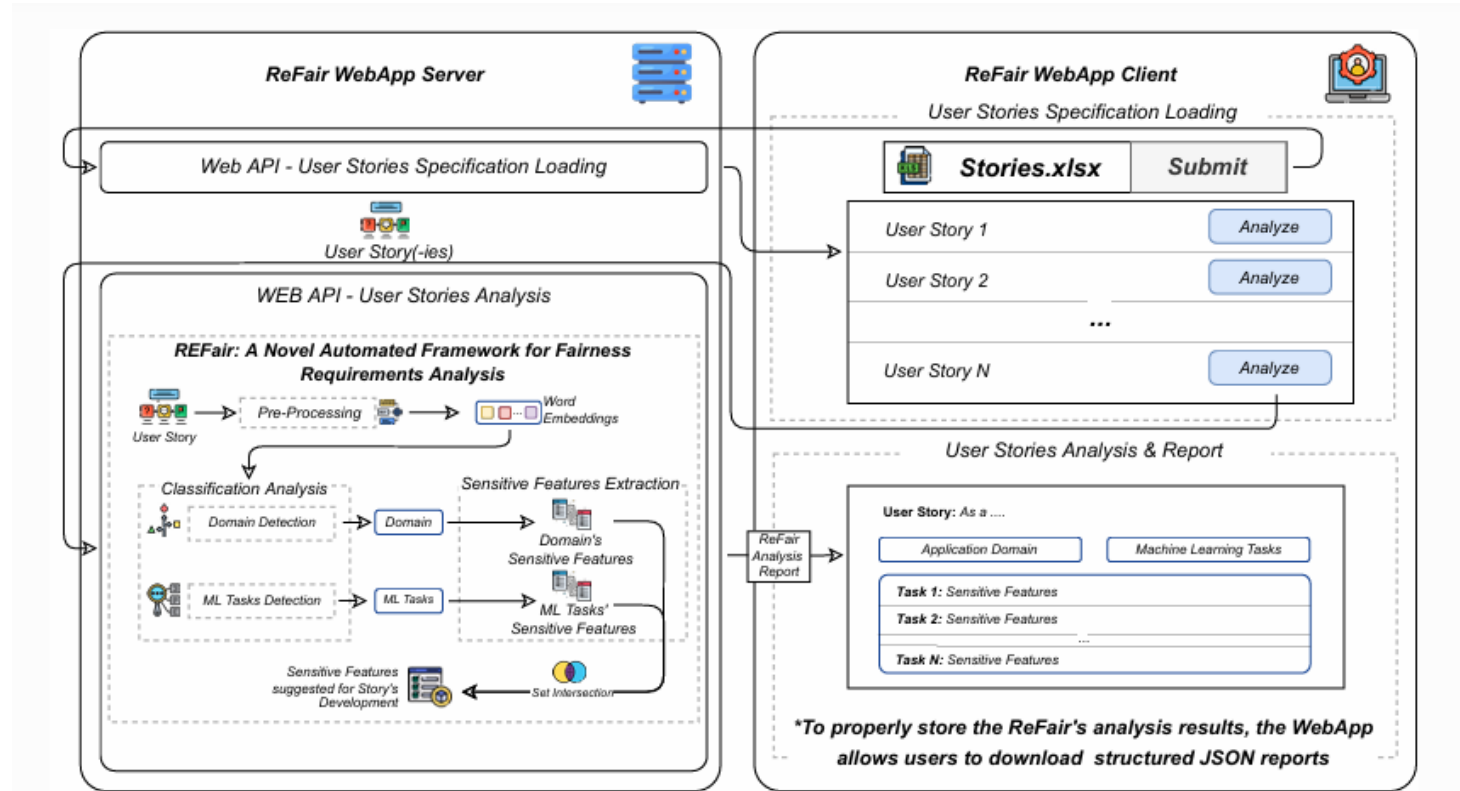
ReFair è un framework automatizzato in grado di raccomandare caratteristiche sensibili da monitorare analizzando le user story.



Successivamente è stata sviluppato un tool di ReFair che incorpora le funzionalità di NLP dell'originale e migliora le prestazioni consentendo l'analisi di più storie utente contemporaneamente e la memorizzazione permanente dei risultati

Introduzione

Architettura pre manutenzione



Il sistema è composto da un **WebApp Server** e un **WebApp Client**. Il server gestisce il caricamento e l'analisi delle user stories tramite API, utilizzando tecniche di **word embedding**, **rilevamento del dominio** e **delle attività di machine learning**, estraendo le caratteristiche sensibili.

Il client consente agli utenti di caricare file **Stories.xlsx**, avviare l'analisi e visualizzare i risultati, che possono essere scaricati in formato **JSON**.

Introduzione

Reverse engineering

Prima di procedere con l'evoluzione di ReFair, è stato fondamentale analizzare nel dettaglio l'architettura esistente.

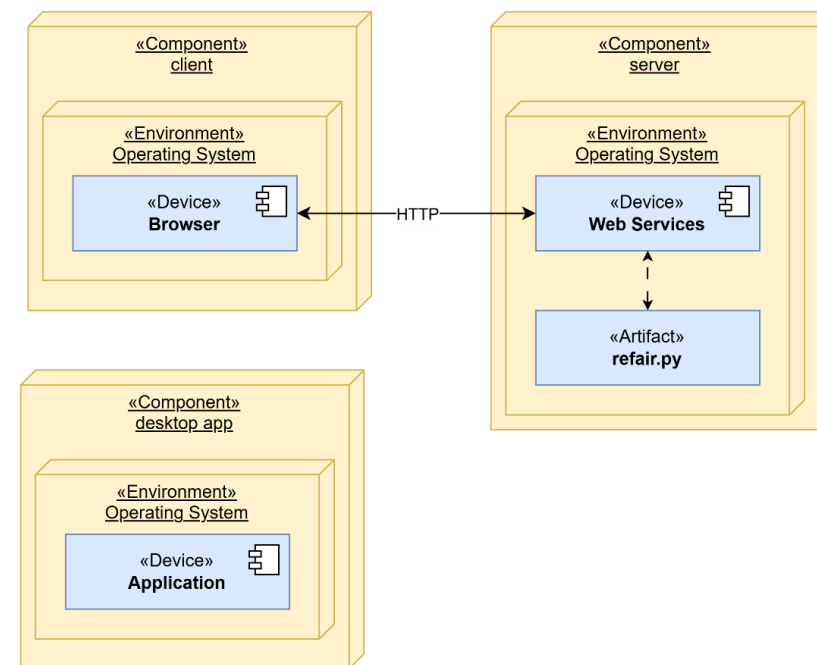
La fase di **Reverse engineering** ci ha permesso di comprendere il funzionamento interno del sistema ed identificare le criticità.

Attraverso il reverse engineering, abbiamo ricostruito l'architettura del sistema e il class diagram, in modo da avere chiaro come funzionasse il sistema internamente.

Introduzione

Reverse engineering

ID	UC AN
Denominazione	Analisi stories.xlsx
Descrizione	Lo UC fornisce la funzionalità di effettuare l'analisi su una user story
Attore principale	Utente
Attori secondari	Webapp
Entry condition	L'utente ha avviato la web app.
Exit condition on success	L'utente visualizza l'analisi della user story
Exit condition on failure	L'utente non visualizza l'analisi della user story
Frequenza stimata	10 volte/gg
Flusso di eventi principale	
1	Utente: L'utente clicca sul pulsante "select file"
2	Sistema: Mostra la finestra per selezionare il file
3	Utente: Seleziona il file stories.xlsx
4	Utente: clicca ok
5	Utente clicca: Load
6	Sistema: il sistema mostra all'utente la lista di tutte le US presenti nel file.xlsx.
7	Utente: Clicca sul pulsante "analyze" vicino alla US che vuole analizzare.
8	Sistema: Il sistema analizza la US.
9	Sistema: Il sistema mostra una finestra con i risultati dell'analisi della US.
Flusso di eventi alternativo: errore nella scelta del file	
2.1	Utente: sceglie un file .json
2.2	Utente: clicca ok
2.3	Utente clicca: Load;
2.4	Sistema: mostra un alert di errore
Flusso di eventi alternativo: errore nella predizione	
8.1	Sistema: il sistema mostra un alert di errore.



Introduzione

Reverse engineering

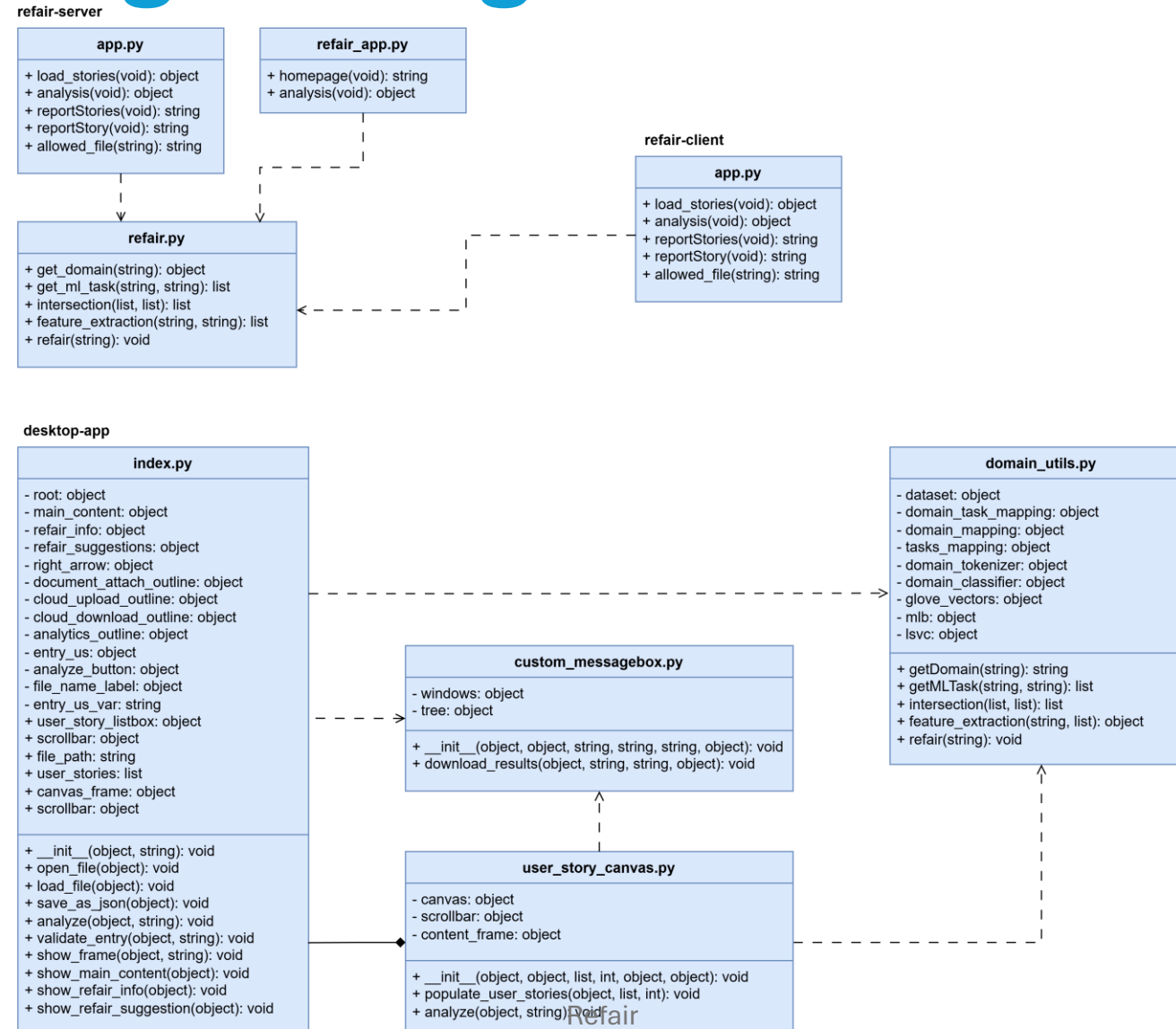
Prima di procedere con l'evoluzione di ReFair, è stato fondamentale analizzare nel dettaglio l'architettura esistente.

La fase di **Reverse engineering** ci ha permesso di comprendere il funzionamento interno del sistema ed identificare le criticità.

Attraverso il reverse engineering, abbiamo ricostruito l'architettura del sistema e il class diagram, in modo da avere chiaro come funzionasse il sistema internamente.

Introduzione

Reverse engineering



Criticità individuate

Dall'analisi condotta durante la fase di **reverse engineering**, supportata dall'estrazione del **Class Diagram**, dell'**Activity Diagram** e del **Sequence Diagram**, sono emerse diverse criticità che influenzavano la scalabilità, la manutenibilità e l'efficienza del sistema.

Architettura Monolitica

Il sistema è costruito con un'architettura **monolitica**, rendendo difficile la **scalabilità e la manutenzione**.

Duplicazione della logica di business tra WebApp e Desktop App

La logica di business è **replicata** su entrambe le piattaforme, aumentando il rischio di **incoerenze** nei risultati.

Maggiore **complessità nella manutenzione**, poiché ogni aggiornamento deve essere applicato a entrambe le implementazioni.

Assenza di Modularità e Riutilizzabilità del Codice

Il codice non è ben **modularizzato**, con molte **funzionalità duplicate** in più parti del sistema.

Difficoltà nel **riutilizzare componenti esistenti**, portando a una crescita disorganizzata del codice.

Change request n°2

Ristrutturazione architettura del sistema

Motivazione

L'attuale architettura del server, **monolitica**, rende il sistema **difficile da scalare, mantenere e aggiornare**. La duplicazione della logica tra WebApp e Desktop App crea **incoerenze e maggiore complessità** nella gestione del codice.

Stato attuale

Attualmente, il sistema è basato su un'architettura monolitica, in cui tutte le funzionalità sono integrate in un unico blocco di codice. Questa struttura rende complessa la manutenzione, limita la scalabilità e rallenta lo sviluppo.

Benefici

Il passaggio ai microservizi migliora la scalabilità, la manutenibilità e la resilienza del sistema, consentendo aggiornamenti e rilasci più rapidi senza impattare l'intera applicazione.

Passi chiave

Identificazione servizi offerti dal server → Sviluppo singoli servizi → Creazione api-gateway → Collegamento desktop app con server → Collegamento web-app con server

Change request n°2

Impact analysis

La Change Request prevede un **refacimento completo dell'architettura del sistema, in particolare del server**, per soddisfare le nuove esigenze **funzionali e non funzionali**, migliorando:

- Scalabilità
- Manutenibilità
- Adattabilità

Questo intervento comporta una **revisione profonda del backend**, nel dettaglio ecco gli insiemi dell'**impact analysis** e le relative metriche.

SIS
refair.py
app.py
refair.vue
index.py
domain_utils.py

CIS
refair.py
app.py
refair.vue
index.py
domain_utils.py

AIS
refair.py
app.py
refair.vue
index.py
domain_utils.py

Metriche
$\text{Recall} = \frac{ CIS \cap AIS }{ AIS } = 1$
$\text{Precision} = \frac{ CIS \cap AIS }{ CIS } = 1$
Inclusiveness = 1

FPIS e DIS non presentano componenti

Change request n°2

Servizi identificati

Ecco descritti i principali microservizi e le loro funzioni all'interno del sistema.



1. Domain Prediction Service

- Analizza le user stories per identificare il **dominio di appartenenza**.
- Utilizza tecniche di **elaborazione del linguaggio naturale (NLP)** per classificare le user stories in base al contesto applicativo.



2. Task Prediction Service

- Identifica le **attività di machine learning** associate a una user story.



3. Report Generation Service

- Identifica le **attività di machine learning** associate a una o più user stories.



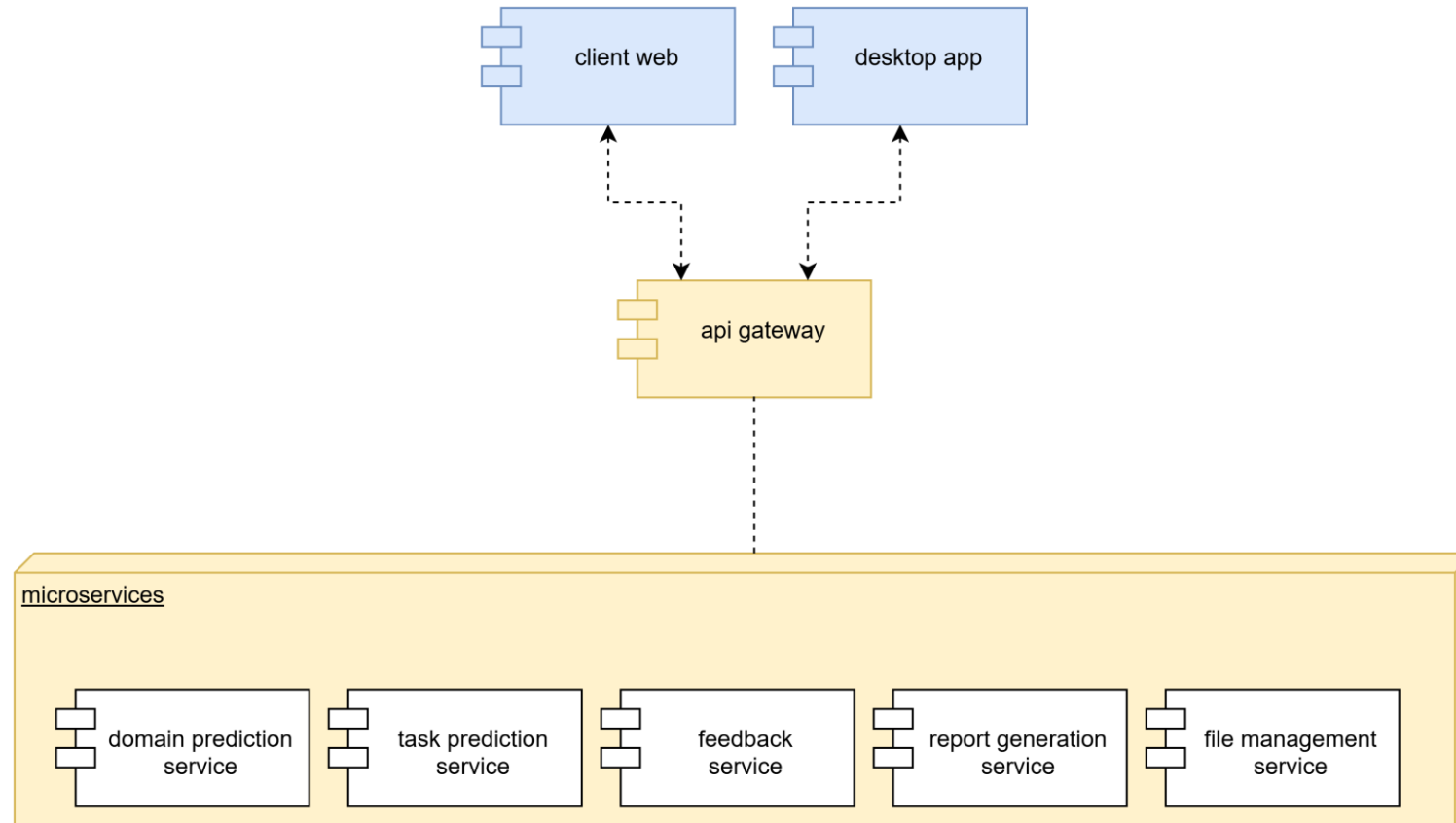
4. File Management Service

- Gestisce il **caricamento, la validazione e la gestione dei file** (es. file .xlsx contenenti user stories).

Change request n°2

Architettura post-modifica

L'architettura è ora composta da diversi microservizi indipendenti, orchestrati da un **API Gateway**, che funge da punto di accesso centralizzato per le richieste provenienti da WebApp e Desktop App. Ogni microservizio è responsabile di una specifica funzionalità, garantendo una gestione più efficiente e modulare del sistema.



Change request n°2

Reachability matrix

A seguito della modifica dell'architettura, si è reso necessario ridisegnare la matrice di raggiungibilità, in quanto l'introduzione di nuovi componenti per la gestione del sistema ha modificato le relazioni e le dipendenze tra i componenti.

Componenti	refair.py	app.py	refairvue	Domain_prediction_service.py	File_management_service.py	Report_generation_service.py	Task_prediction_service.py	Api_gateway.py	index.py	domain_utils.py	user_story_canvas.py	custom_messagebox.py
refair.py		1										
app.py	1											
refair.vue				2	2	2	2	1				
Domain_prediction_service.py			2			1		1	1			
File_management_service.py			2					1	1			
Report_generation_service.py			2	1			1	1	1			
Task_prediction_service.py			2			1		1	1			
Api_gateway.py			1	1	1	1	1		1			
index.py				2	2	2	2	1				
domain_utils.py												
user_story_canvas.py									1	2		1
custom_messagebox.py									1	2	1	

Refair

Change request n°2

Considerazioni finali CR2

La ristrutturazione dell'architettura ha portato significativi miglioramenti, trasformando il sistema da un'architettura **monolitica** a una basata su **microservizi modulari**.

Aspetto	Prima	Dopo
Scalabilità	Difficile e rigida. Tutto il codice è centralizzato in un unico blocco, difficile da espandere.	Modulare e adattabile. I microservizi possono essere scalati indipendentemente.
Manutenzione	Complessa e lenta. Ogni aggiornamento o modifica richiede individuazione della parte interessata, difficile visto la struttura del server.	Più semplice e indipendente. Le modifiche possono essere fatte su singoli servizi senza impattare l'intero sistema.
Coerenza	Incoerenze tra Web e Desktop App, con logiche duplicate	Stessa logica per entrambe le piattaforme, grazie alla centralizzazione della logica di business nel backend.

Change request n°1

Implementazione di HITL

Motivazione

Per migliorare l'**accuratezza dei modelli predittivi**, è stato introdotto un meccanismo di **Human-In-The-Loop (HITL)**. Questo approccio permette agli utenti di **valutare i risultati** generati dal sistema, fornendo un feedback attivo.

L'obiettivo è affinare progressivamente i modelli di machine learning, riducendo gli errori e migliorando la qualità delle predizioni.

Stato attuale

Attualmente, il sistema utilizza **modelli di machine learning statici**, basati su un dataset pre-addestrato. Senza aggiornamenti e retraining periodici, c'è il rischio di **diminuzione dell'accuratezza**, soprattutto con dati o scenari non previsti. Inoltre, manca un **meccanismo di raccolta dei feedback** dagli utenti.

Benefici

Migliore accuratezza da parte del sistema grazie all'addestramento continuo da parte dei modelli di machine learning.

Maggiore capacità del sistema di adattarsi a nuovi scenari o variazioni nei dati.

Change request n°1

Impact analysis

Procediamo ora con l'**Impact Analysis** per identificare le componenti del sistema coinvolte e valutare gli effetti derivanti dall'implementazione di questa modifica.

SIS
Refair.vue
User_story_canvas.py
Api_gateway.py

CIS
Refair.vue
User_story_canvas.py
Api_gateway.py
Domain_prediction_service.py
File_management_service.py
Report_generation_service.py
Task_prediction_service.py
Index.py

AIS
Refair.vue
User_story_canvas.py
Api_gateway.py

FPIS
Domain_prediction_service.py
File_management_service.py
Report_generation_service.py
Task_prediction_service.py
Index.py

Metriche
Recall = $\frac{ CIS \cap AIS }{ AIS } = 1$
Precision = $\frac{ CIS \cap AIS }{ CIS } = 0.375$
Inclusiveness = 1

DIS non contiene elementi

Change request n°1

Modifiche apportate

Per integrare il meccanismo di Human-In-The-Loop (HITL) nel sistema, sono stati implementati diversi componenti. I seguenti passi descrivono le principali modifiche apportate.

- ✓ **1. Introduzione di un nuovo servizio per la gestione dei feedback**
Creazione di un **microservizio dedicato** alla raccolta e archiviazione dei feedback utente.
Implementazione di API per la comunicazione tra il sistema e il servizio di feedback.
- ✓ **2. Implementazione di un'interfaccia utente per il feedback**
Aggiunta di una **sezione dedicata nell'UI** per consentire agli utenti di valutare l'accuratezza dei risultati.
- ✓ **3. Servizio di retraining del modello basato sul feedback raccolto**
Il sistema attiva il **retraining del modello** dopo un numero predefinito di feedback ricevuti.
Il modello aggiornato viene **valutato con metriche di performance**, e sostituisce il precedente **solo se mostra miglioramenti rispetto alla versione attuale**.

Change request n°1

Considerazioni finali CR1

L'implementazione del meccanismo HITL ha trasformato il sistema in una soluzione più dinamica, precisa e adattabile, migliorando significativamente le performance predittive.

Aspetto	Prima	Dopo
Coinvolgimento Utenti	Nessuna interazione diretta con il sistema predittivo.	Gli utenti forniscono feedback attivo , contribuendo a migliorare il modello.
Accuratezza modelli	Basata esclusivamente su un dataset pre-addestrato, senza aggiornamenti dinamici.	Il feedback utente permette il miglioramento continuo e l'adattamento ai nuovi dati.
Apprendimento modelli	Statico, senza meccanismi di retraining automatico.	Il retraining avviene periodicamente e solo se le metriche del sistema sono migliorate.

Change request n°3

Refactoring del codice

Motivazione

L'attuale codice sorgente **presenta problemi di leggibilità e manutenibilità**, rendendo complesso l'approccio per nuovi sviluppatori e la gestione di future evoluzioni. L'assenza di documentazione e la presenza di classi/metodi non utilizzati complicano l'interpretazione del codice. Il refactoring mira a **semplificare il codice, applicare le best practice** e garantire maggiore conformità ai principi **SOLID**.

Stato attuale

Mancano informazioni sui parametri richiesti e sugli output generati.

Difficile comprendere il funzionamento di classi e metodi.

Scarsa leggibilità, dovuta alla mancanza di commenti e spiegazioni nel codice.

Benefici

Codice più pulito, conforme alle best practice.

Eliminazione di codice obsoleto, con una base più efficiente e leggera.

Applicazione dei principi di programmazione SOLID

Change request n°3

Impact analysis

L'attività di **refactoring del codice** impatta sull'intero sistema.

SIS
refair.py
app.py
refair.vue
Domain_prediction_service.py
File_management_service.py
Report_generation_service.py
Task_prediction_service.py
Api_gateway.py
index.py
domain_utils.py
user_story_canvas.py
custom_messagebox.py

CIS
refair.py
app.py
refair.vue
Domain_prediction_service.py
File_management_service.py
Report_generation_service.py
Task_prediction_service.py
Api_gateway.py
index.py
domain_utils.py
user_story_canvas.py
custom_messagebox.py

AIS
refair.py
app.py
refair.vue
Domain_prediction_service.py
File_management_service.py
Report_generation_service.py
Task_prediction_service.py
Api_gateway.py
index.py
domain_utils.py
user_story_canvas.py
custom_messagebox.py

Metriche
$\text{Recall} = \frac{ CIS \cap AIS }{ AIS } = 1$
$\text{Precision} = \frac{ CIS \cap AIS }{ CIS } = 1$
Inclusiveness = 1

FPIS e DIS non contengono elementi

Change request n°3

Modifiche apportate

Il codice del sistema presentava un forte accoppiamento tra le varie componenti, presentava criticità in termini di leggibilità e ciò rendeva complessa la sua comprensione. Per migliorare il codice del progetto sono state applicate le best practice e i principi SOLID.

Sigla	Descrizione
SRP	Single responsibility principle Ogni classe dovrebbe avere una ed una sola responsabilità.
OCP	Open/closed principle Un'entità software dovrebbe essere aperta alle estensioni, ma chiusa alle modifiche.
LSP	Liskov substitution principle Gli oggetti dovrebbero poter essere sostituiti con dei loro sottotipi, senza alterare il comportamento di chi li utilizza.
ISP	Interface segregation principle Sarebbero preferibili più interfacce specifiche.
DIP	Dependency inversion principle Una classe dovrebbe dipendere dalle astrazioni, non da classi concrete.

	Domain Prediction Service	Task Prediction service	File Management service	Report Generation service	Feedback service
SRP	X	X	X	X	X
OCP	V	V	V	V	V
LSP	V	V	V	V	V
ISP	Non applicabile	Non applicabile	Non applicabile	Non applicabile	Non applicabile
DIP	X	X	V	X	X

Change request n°3

Modifiche apportate

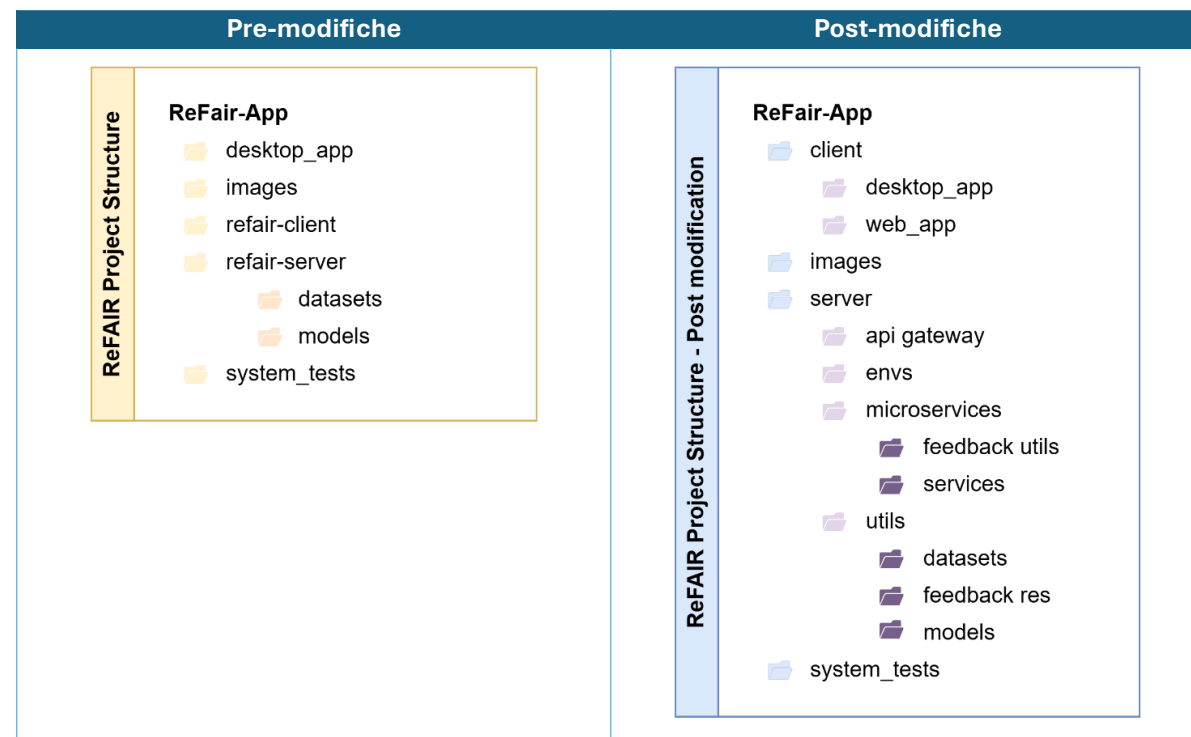
Di seguito sono riportate tutte le modifiche necessarie per il completamento della CR3.

- ✓ Ogni microservizio è stato **riprogettato secondo i principi SRP, OCP, LSP, DIP**
- ✓ Uso di **interfacce e astrazioni** per ridurre l'accoppiamento tra i componenti.
- ✓ Implementazione di **dipendenze più flessibili**, facilitando la manutenzione e l'evoluzione del sistema.
- ✓ Ogni **metodo principale è stato documentato**, specificando parametri, output e comportamento atteso.
- ✓ **Struttura dei file e dei moduli ottimizzata** per rendere il codice più leggibile e manutenibile.
- ✓ Eliminazione di **ridondanze e codice obsoleto**, migliorando l'efficienza del sistema.

Change request n°3

Modifiche apportate

	Domain Prediction Service	Task Prediction service	File Managemen t service	Report Generation service	Feedback service
SRP	V	V	V	V	V
OCP	V	V	V	V	V
LSP	V	V	V	V	V
ISP	Non applicabile	Non applicabile	Non applicabile	Non applicabile	Non applicabile
DIP	V	V	V	V	V



Change request n°3

Considerazioni finali CR3

L'attività di **refactoring del codice** ha migliorato significativamente la qualità del software, rendendolo **più leggibile, manutenibile e scalabile**.

Aspetto	Prima	Dopo
Commenti	Assenti o insufficiente, difficile capire il funzionamento del codice.	Aggiunti commenti dettagliati e documentazione per classi e metodi.
Manutenibilità	Struttura del codice complessa e difficile da aggiornare.	Codice più leggibile, organizzato e conforme alle best practice.
Applicazione dei Principi SOLID	Assenza di una chiara separazione delle responsabilità tra i moduli, con codice accoppiato e difficile da estendere.	Implementati i principi SOLID , garantendo una struttura più modulare, flessibile e facilmente estendibile per futuri sviluppi.

Testing

Testing

I test sono stati fondamentali per garantire la qualità e la stabilità del sistema. Sono stati eseguiti test prima della manutenzione, dopo ogni chiusura di Change Request e alla fine del progetto, assicurandoci che ogni modifica fosse correttamente integrata e non introducesse regressioni.

Tecnologie utilizzate

Pytest

Pytest cov

Selenium

Test eseguiti

- ◆ Test di unità
- ◆ Test di integrazione
- ◆ Test di sistema
- ◆ Test di regressione

Testing

Unit testing

Sul progetto Refair è stato eseguito unit test su tutte le nuove classi inserite, verificandone il corretto funzionamento e riducendo il rischio di bug nelle fasi successive di sviluppo.

Metodologia applicata: **White-box con branch coverage**

Test Case	Funzione Testata	Input	Oracolo
test_feedback_domain_success	POST /feedback/domain	JSON con "user_story", "predicted_domain" e "feedback_value" validi	status: success, message: "Domain feedback received", HTTP 200
test_feedback_tasks_success	POST /feedback/tasks	JSON con "user_story", "domain", "predicted_tasks" e "feedback_value" validi	status: success, message: "Task feedback received", HTTP 200
test_feedback_no_json	POST /feedback/domain	Dati non in formato JSON ("Invalid Text")	HTTP 415
test_feedback_missing_fields	POST /feedback/domain	JSON senza i campi obbligatori (es. solo "user_story")	status: success, HTTP 200 (potenziale bug, dovrebbe essere errore)

```
def test_feedback_tasks_success(client, mocker):
    """ Testa l'invio di feedback valido per le task """
    mocker.patch("services.feedback_manager.FeedbackManager.save_feedback", return_value=None)

    response = client.post('/feedback/tasks', data=json.dumps({
        "user_story": "As a transportation planner, I want to use inverse reinforcement learning to understand the underlying motivation",
        "domain": "Transportation",
        "predicted_tasks": ["pricing"],
        "feedback_value": 5
    })), content_type='application/json')

    assert response.status_code == 200
    assert response.json['status'] == 'success'
    assert response.json['message'] == "Task feedback received"

    assert response.status_code == 200
    assert response.json['status'] == 'success'
    assert response.json['message'] == "Task feedback received"
```

Testing

Unit testing

Sul progetto Refair è stato eseguito unit test su tutte le nuove classi inserite, verificandone il corretto funzionamento e riducendo il rischio di bug nelle fasi successive di sviluppo.

Metodologia applicata: **White-box con branch coverage**

```
def test_predict_domain_success(client, mocker):
    """ Testa la predizione con una user story valida """
    mocker.patch("server.microservices.domain_prediction_service.domain_classifier.p

    response = client.post('/predidef test_predict_domain_success(client, mocker):
    """ Testa la predizione con una user story valida """
    mocker.patch("server.microservices.domain_prediction_service.domain_classifier.p

    response = client.post('/predict/domain', data=json.dumps({
        "user_story": "As a cardiologist, I want to identify risk factors for heart disease."
    })), content_type='application/json')

    assert response.status_code == 200
    assert response.json['status'] == 'success'
    assert response.json['domain'] == "Cardiology"ct/domain', data=json.dumps({
        "user_story": "As a cardiologist, I want to identify risk factors for heart disease."
    })), content_type='application/json')

    assert response.status_code == 200
    assert response.json['status'] == 'success'
    assert response.json['domain'] == "Cardiology"
```

Test case	Funzione testata	Input	oracolo
test_predict_domain_success	POST /predict/domain	JSON con "user_story": "As a cardiologist, I want to identify risk factors for heart disease."	status: success, domain: "Cardiology", HTTP 200
test_predict_domain_no_json	POST /predict/domain	Dati non in formato JSON ("Not a JSON")	status: failure, motivation: "Request body must be JSON", HTTP 400
test_predict_domain_missing_user_story	POST /predict/domain	JSON senza il campo "user_story"	status: failure, motivation: "Missing 'user_story' in request", HTTP 400
test_predict_domain_classifier_error	POST /predict/domain	JSON con "user_story": "As a software engineer, I want to automate testing."	Errore interno del modello, HTTP 400

Testing

Integration testing

Per il test di integrazione è stato utilizzato un approccio bottom-up, in particolare ci siamo soffermati sull'integrazione di tutti i servizi offerti dal sistema, con l'api gateway. Anche in questo caso è stata testata l'integrazione delle nuove componenti del sistema.

Approccio utilizzato: **incrementale bottom-up**

```
def test_feedback_domain_success(client):
    """ Testa l'invio di un feedback valido per il dominio """

    data = {def test_feedback_domain_success(client):
        """ Testa l'invio di un feedback valido per il dominio """

    data = {
        "user_story": "As an orthopedic surgeon, I want to review X-ray images to identify bone fractures quickly.",
        "predicted_domain": "orthopedics",
        "feedback_value": 4
    }

    response = client.post(f"{API_GATEWAY_URL}/feedback/domain", json=data)

    print("Response Status:", response.status_code)
    print("Response JSON:", response.json())

    assert response.status_code == 200, f"Errore HTTP: {response.status_code} - {response.text}"
    json_response = response.json()
    assert json_response["status"] == "success"
    assert json_response["message"] == "Domain feedback received"
    "user_story": "As an orthopedic surgeon, I want to review X-ray images to identify bone fractures quickly.",
    "predicted_domain": "orthopedics",
    "feedback_value": 4
}

response = client.post(f"{API_GATEWAY_URL}/feedback/domain", json=data)

print("Response Status:", response.status_code)
print("Response JSON:", response.json())

assert response.status_code == 200, f"Errore HTTP: {response.status_code} - {response.text}"
json_response = response.json()
assert json_response["status"] == "success"
assert json_response["message"] == "Domain feedback received"
```

Nome Test	Input	Oracolo (Output Atteso)
test_feedback_domain_success	{ "user_story": "As an orthopedic surgeon, I want to review X-ray images to identify bone fractures quickly.", "predicted_domain": "orthopedics", "feedback_value": 4 }	status: success, message: "Domain feedback received"
test_feedback_tasks_success	{ "user_story": "As a cardiologist, I want to identify multiword expressions in patient notes to identify risk factors for heart disease.", "domain": "cardiology", "predicted_tasks": ["Identify Multiword Expressions", "Risk Factor Analysis"], "feedback_value": 5 }	status: success, message: "Task feedback received"
test_feedback_invalid_json	Corpo richiesta con data="invalid data", header Content-Type: text/plain	status: failure, motivation: "Request body must be JSON"

Testing

Integration testing

Per il test di integrazione è stato utilizzato un approccio bottom-up, in particolare ci siamo soffermati sull'integrazione di tutti i servizi offerti dal sistema, con l'api gateway. Anche in questo caso è stata testata l'integrazione delle nuove componenti del sistema.

Approccio utilizzato: **incrementale bottom-up**

Nome Test	Input	Oracolo
test_predict_domain_success	{ "user_story": "As a cardiologist, I want to identify multiword expressions in patient notes to identify risk factors for heart disease." }	status: success, domain: stringa non vuota
test_predict_domain_invalid_json	Corpo richiesta con data="invalid data", header Content-Type: text/plain	status: failure, motivation: "Request body must be JSON"
test_predict_domain_missing_user_story	{ "user_story": "" } (campo vuoto)	status: failure, motivation: "Missing 'user_story'"

```
def test_predict_domain_success(client):
    """ Testa la predizione del dominio con una user story valida """

    data = {
        "user_story": "As a cardiologist, I want to identify multiword expressions in patient notes to identify risk factors for heart disease."
    }

    def test_predict_domain_success(client):
        """ Testa la predizione del dominio con una user story valida """

        data = {
            "user_story": "As a cardiologist, I want to identify multiword expressions in patient notes to identify risk factors for heart disease."
        }

        response = client.post(f"{API_GATEWAY_URL}/predict/domain", json=data)

        print("Response Status:", response.status_code)
        print("Response JSON:", response.json())

        assert response.status_code == 200, f"Errore HTTP: {response.status_code} - {response.text}"
        json_response = response.json()
        assert json_response["status"] == "success"
        assert isinstance(json_response["domain"], str) and len(json_response["domain"]) > 0

        response = client.post(f"{API_GATEWAY_URL}/predict/domain", json=data)

        print("Response Status:", response.status_code)
        print("Response JSON:", response.json())

        assert response.status_code == 200, f"Errore HTTP: {response.status_code} - {response.text}"
        json_response = response.json()
        assert json_response["status"] == "success"
        assert isinstance(json_response["domain"], str) and len(json_response["domain"]) > 0
```

Testing

System testing

Per il test di sistema è stata utilizzata la suite di test pre-esistente, sono stati aggiunti test per testare la funzionalità aggiunta «Feedback».

L'approccio utilizzato per i test di sistema, è il **category partition**, soffermandosi principalmente sulla funzionalità «**Load**».

Funzionalità testate:

- **Load**
- **Report**
- **Analyze**
- **Feedback**

Testing

System testing

Funzionalità testate: **Load**

ID	Categoria	Scelte
FN	File Name	Nome file: 1. uguale a stories; 2. diverso da stories.
FE	File Extension	Estensione del file: 1. uguale axlsx [property EXCEL]; 2. diverso da xlsx [error].
.	.	.
.	.	.
.	.	.
NS	Number of Sheets	Numero di fogli: 1. =1 [if EXCEL] 2. >1 [if EXCEL] [property MULTIP

ID	Test Frame	Oracolo
LOAD_TC_1	FN2, FE1, NS1, NCFS2, CNCOS0, CHFS2, CHOS0, NUSFS2, NUSOS0, SFFS3, SFOS0	[success]
LOAD_TC_2	FN1, FE2, NS0, NCFS0, CNCOS0, CHFS0, CHOS0, NUSFS0, NUSOS0, SFFS0, SFOS0	[error] This type of file is not supported. Upload an xlsx file.
LOAD_TC_3	FN1, FE1, NS1, NCFS1, CNCOS0, CHFS0, CHOS0, NUSFS0, NUSOS0, SFFS0, SFOS0	[error] No column "User Story" found.

Testing

System testing

Funzionalità testate: **Feedback**

ID	Categoria	Scelte
VD	Voto dominio	1. Voto compreso tra 1 e 5 2. Nessun voto [error]
VT	Voto task	1. Voto compreso tra 1 e 5 2. Nessun voto [error]

ID	Test frame	Oracolo
Feedback_TC_1	VD1, VT1	Success
Feedback_TC_2	VD1, VT2	Error
Feedback_TC_3	VD2, VT2	Error
Feedback_TC_4	VD2, VT1	Error

Testing

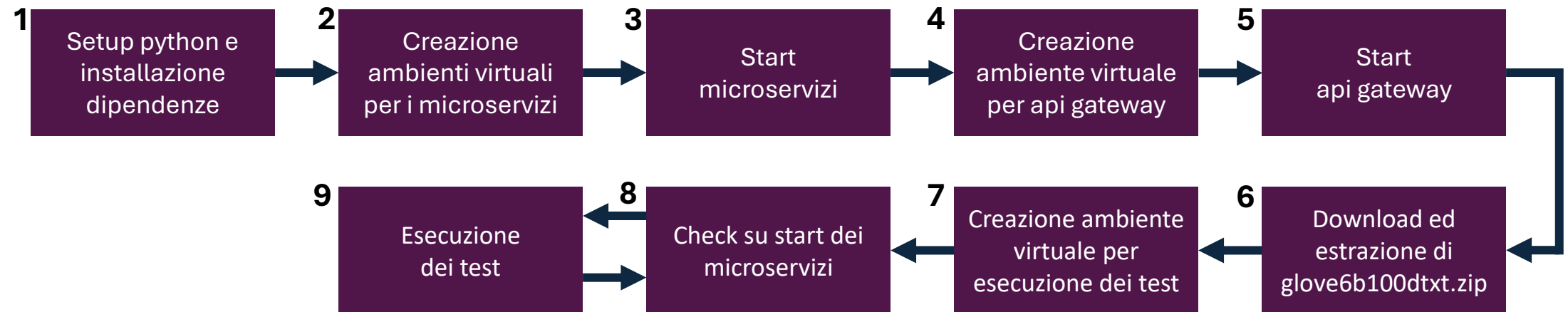
Regression testing

Con il test di regressione vogliamo verificare che modifiche, correzioni di bug o nuove funzionalità non abbiano impatti negativi su parti già testate e funzionanti.

Strategia utilizzata:

test-all

Approccio più dispendioso in termini di tempo, ma garantisce massima copertura e affidabilità.



Quando viene eseguita una push sul branch dev, un workflow avvia automaticamente l'intera suite di test. Riduce il rischio di rilasciare modifiche non testate e previene regressioni su componenti non direttamente coinvolte.

Next

Sviluppi futuri

Le modifiche apportate all'applicazione ReFair hanno permesso di superare varie criticità predisponendo l'intero ecosistema a nuovi sviluppi.

- 1.** **Open Source**
Superare il vincolo dei file .xlsx, estensione proprietaria, in favore di tracciati open come ad esempio i .csv.
- 2.** **Estensione delle capacità**
Estendere domini e caratteristiche sensibili riconoscibili dal classificatore, in modo da rendere il progetto più completo e robusto.
- 3.** **Persistenza dei dati**
Collegare un database per salvare i risultati, consentendo magari all'utente di mantenere uno storico delle estrazioni fatte.

FINE

Grazie per l'attenzione!